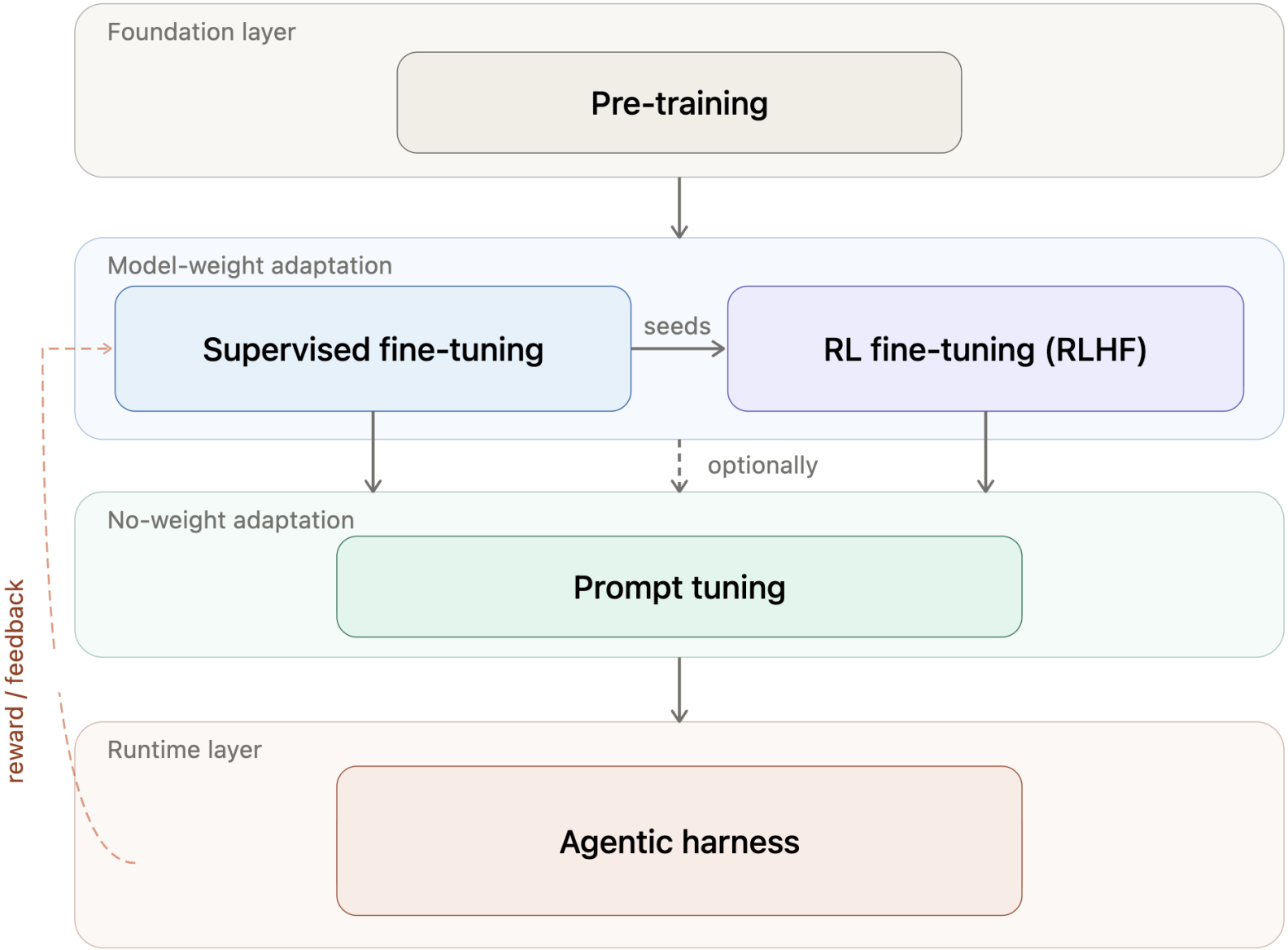
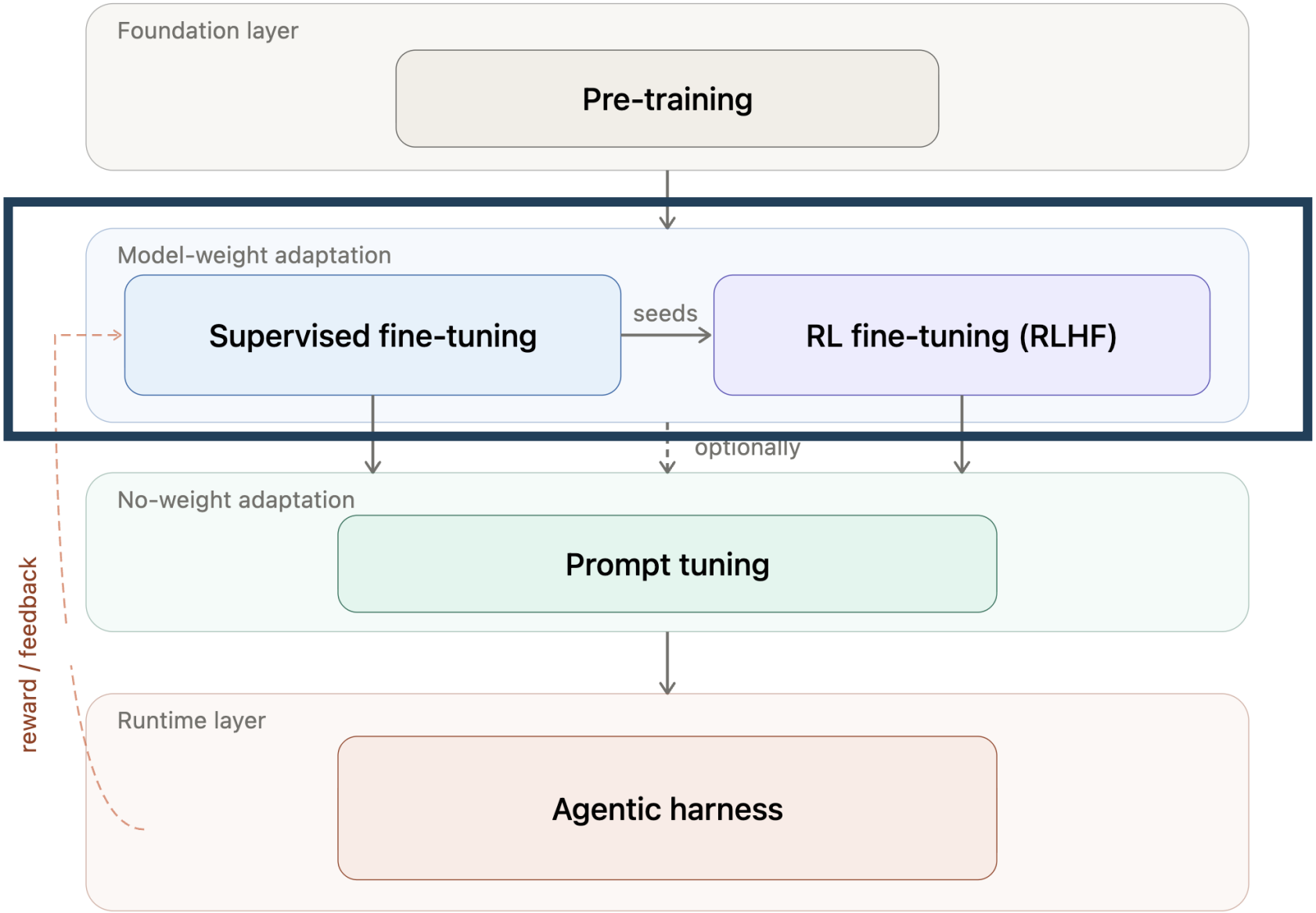
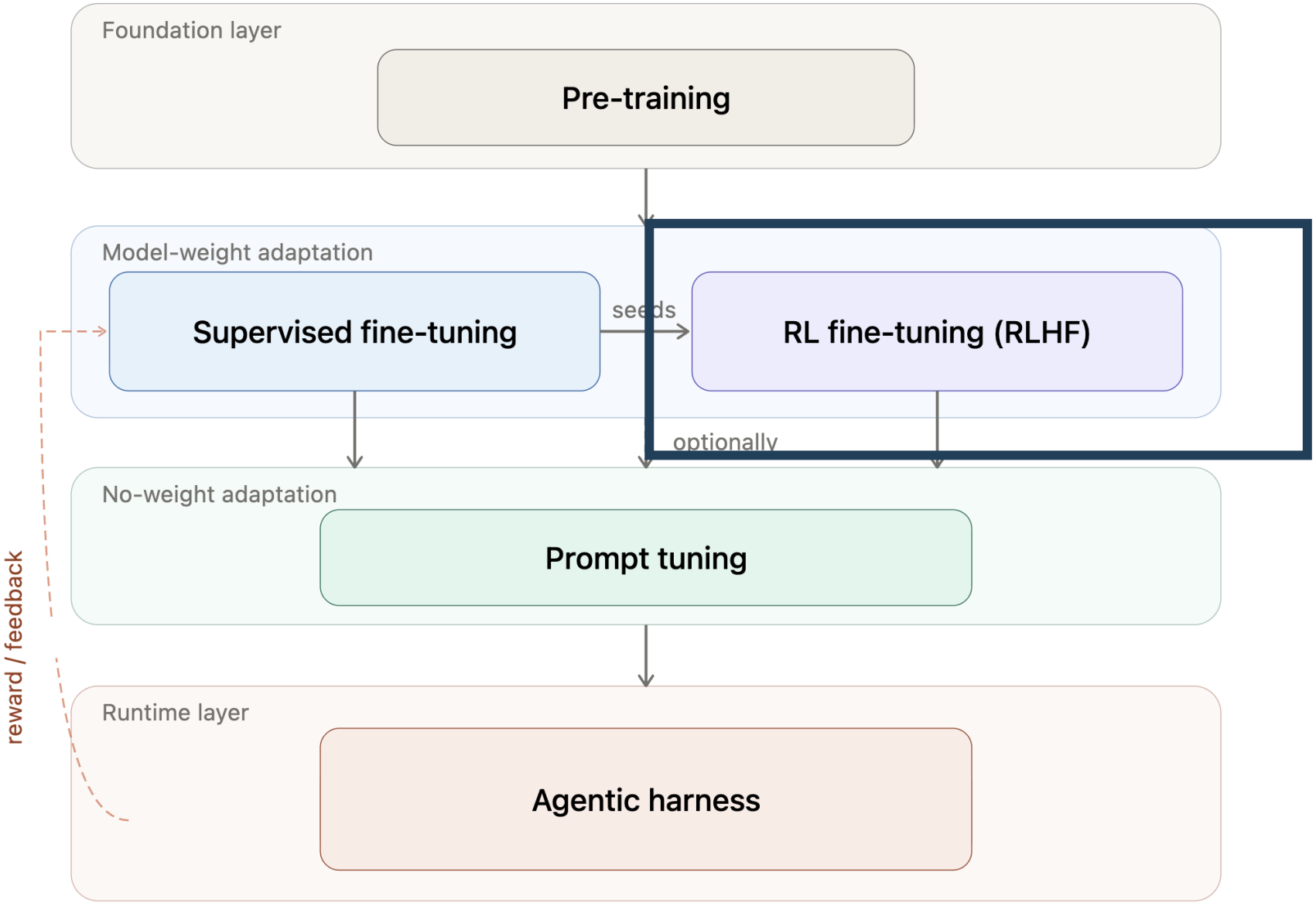
Lecture 10: RLHF & Preference Ranking

Kai-Wei Chang
CS @ UCLA
kw@kwchang.net

Credit: some slides are modified from Vivek Srikumar, Jason Eisner, Chris Manning


Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
reward / feedback
Runtime layer
Agentic harness


Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
reward / feedback
Runtime layer
Agentic harness


Foundation layer
Pre-training
Model-weight adaptation
Supervised fine-tuning
seeds
RL fine-tuning (RLHF)
optionally
No-weight adaptation
Prompt tuning
reward / feedback
Runtime layer
Agentic harness

RLHF – Human Preferences

- Which of these two responses is more helpful?

What are the steps for making a simple cake?

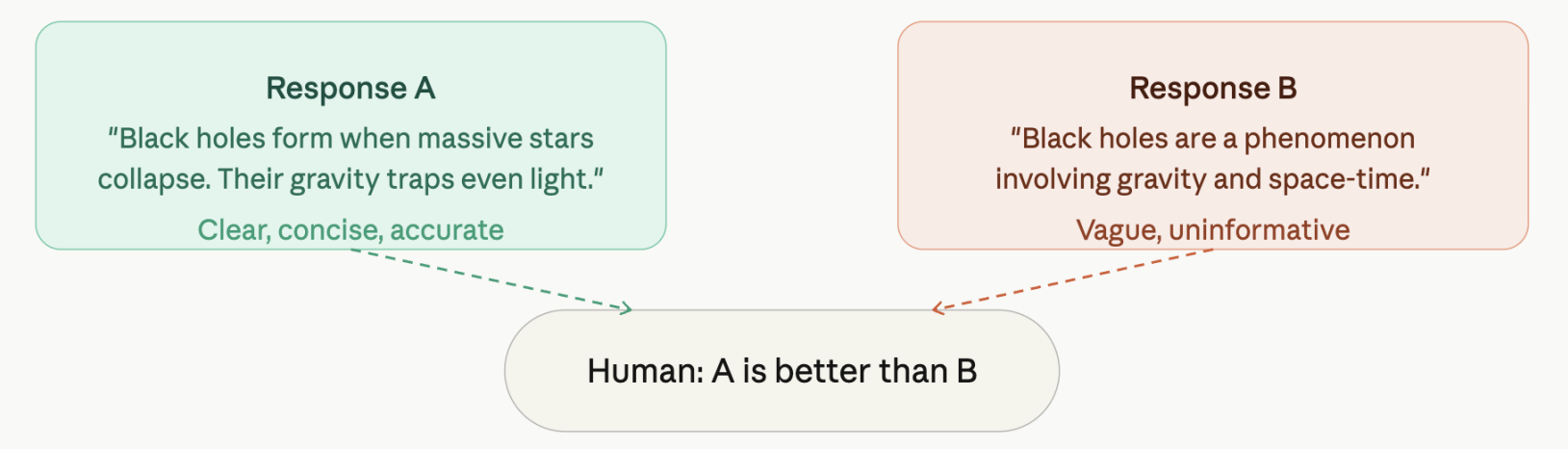
1. *Warm up the oven.*
2. *Grease a cake pan.*
3. *Blend dry ingredients in a bowl.*
4. *Incorporate butter, milk, and vanilla.*
5. *Mix in the eggs.*
6. *Pour into the prepared pan.*
7. *Bake until golden brown.*
8. *Add frosting if desired.*

What are the steps for making a simple cake?

1. *Preheat oven to 350°F (175°C).*
2. *Grease and flour a cake pan.*
3. *In a bowl, combine 2 cups flour, 1.5 cups sugar, 3.5 tsp baking powder, and a pinch of salt.*
4. *Add 1/2 cup butter, 1 cup milk, and 2 tsp vanilla; mix well.*
5. *Beat in 3 eggs, one at a time.*
6. *Pour batter into the pan.*
7. *Bake for 30-35 minutes or until a toothpick comes out clean.*
8. *Let cool, then frost or serve as desired.*

RLHF – Human Preferences

- Instead of evaluating a single example
  - Sample two outputs for the same input from the model
  - And choose a winner



RLHF – Human Preferences

- Instead of evaluating a single example
  - Sample two outputs for the same input from the model
  - And choose a winner
- Formally, we get a dataset of inputs $\boldsymbol{x}^{\{i\}}$ paired with a winning output $\boldsymbol{y}_w^{(i)}$ and a losing output $\boldsymbol{y}_l^{(i)}$

$$\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^{N}$$

RLHF -- Learning

- Now we have a dataset of inputs $\boldsymbol{x}^{\{i\}}$ paired with a winning output $\boldsymbol{y}_w^{(i)}$ and a losing output $\boldsymbol{y}_l^{(i)}$

$$\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^{N}$$

- We want to learn to generate outputs $\mathbf{y}^{(i)}$ given inputs $\mathbf{x}^{(i)}$
- How do we learn from this data?
  - Can we just pretend $\boldsymbol{y}_w^{(i)}$ are annotated outputs?
  - Do we just throw $\boldsymbol{y}_l^{(i)}$ away?

RLHF -- Learning

- Now we have a dataset of inputs $\boldsymbol{x}^{\{i\}}$ paired with a winning output $\boldsymbol{y}_w^{(i)}$ and a losing output $\boldsymbol{y}_l^{(i)}$

$$\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^{N}$$

- We want to learn to generate outputs $\mathbf{y}^{(i)}$ given inputs $\mathbf{x}^{(i)}$
- Use this data to learn a model to score outputs
  - Good outputs → high score, bad outputs → low score
  - This will be our reward model
- Use this model in reinforcement learning to fine-tune your LM

Reinforcement Learning
-- A brief introduction

- There are various RL methods
  - REINFORCE (a simple gradient-based algorithm)
  - Proximal policy optimization (PPO)
  - Direct Preference Optimization (DPO)

Reinforcement Learning

- There are various RL methods
- The most common nowadays are ***policy gradient methods***
- Maximize some performance measure via gradient ascent
- One of the simplest algorithms to do this is REINFORCE [Williams 1992]

The REINFORCE Algorithm

- REINFORCE is a straight forward derivation of the value function objective
- While it gives an objective that looks very similar to log-likelihood, it is fundamentally different — this is not about data likelihood!

The REINFORCE Algorithm

- Input: differentiable parameterized policy $\pi_\theta(a \mid s)$
- Output: parameters θ
- Hyper-parameters: step size α
- Optimizing

$$\theta = \arg\max_\theta v_{\pi_\theta}(s_0)$$

While true:

For $t = 0, \ldots, T$ steps:

$a_t \sim \pi_\theta(a \mid s_t)$

$s_{t+1} \leftarrow T(s_t, a_t)$

$r_t \leftarrow r(s_t, a_t)$

For $t = 0, \ldots, T$ steps:

$G \leftarrow \sum_{k=t}^{T} r_t$

$\theta \leftarrow \theta + \alpha G \nabla \ln \pi_\theta(a_t \mid s_t)$

- **State** s_t → current text (prompt + tokens so far)
- **Action** a_t → next token
- **Policy** $\pi_\theta(a_t \mid s_t)$ → probability distribution over the vocabulary
- **Transition** $T(s_t, a_t)$ → append that token to the sequence
- **Reward** r_t → scalar feedback (usually from a reward model; often given at end of sequence

The REINFORCE Algorithm -- Intuition

- Given the same s_0, how do a_t, s_t, and r_t differ between each round of the outside loop?
- When is G > 0?
- If G > 0, what happens to the probability immediately after th update?
- And if G < 0?

While true:

For $t = 0, \ldots, T$ steps:

$a_t \sim \pi_\theta(a \mid s_t)$

$s_{t+1} \leftarrow T(s_t, a_t)$

$r_t \leftarrow r(s_t, a_t)$

For $t = 0, \ldots, T$ steps:

$G \leftarrow \sum_{k=t}^{T} r_t$

$\theta \leftarrow \theta + \alpha G \nabla \ln \pi_\theta(a_t \mid s_t)$

RLHF for LLMs

- An action space the size of the vocabulary is huge
  - But the output sequence space is even larger
  - Makes the value function hard to evaluate

REINFROCE vs MLE

- Gradient of MLE

$$\nabla_\theta \mathcal{L}_{\text{MLE}} = -\sum_t \nabla_\theta \log \pi_\theta(a_t^{\text{data}} \mid s_t)$$

- Gradient of Reinforce

$$\nabla_\theta J(\theta) = \mathbb{E}\left[G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

- MLE is a special case when

$$G_t = 1 \quad \text{for correct token, 0 otherwise}$$

RLHF for LLMs -- The Reward Model

- RL requires a reward function $r : S \times A \rightarrow \mathbb{R}$
- In the LLM formulation we just introduced: the states and actions are just text, including the prompt and the output completion
- We are going to learn the reward function, so it's parametrized by ψ: $r_\psi([x; y]) \rightarrow \mathbb{R}$
- Our data: inputs paired with a winning output and a losing output: $\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^N$
- How do we get the reward function from this data?

Reward Model: Bradley-Terry Model

- Goal: estimate the parameter ψ for the reward model r_ψ such that $r_\psi([\boldsymbol{x}; \boldsymbol{y}]) \rightarrow \mathbb{R}$
- Data: inputs paired with a winning output and a losing output: $\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^N$
- The Bradley-Terry Model connects scores s(.) to preferences $\succ$

$$p(a \succ b) = \sigma(s(a) - s(b)) \qquad \sigma(x) = \frac{1}{1 + e^{-x}}$$

- If we can recover these scores, we can just use them as rewards.

Reward Model: Bradley-Terry Model

- We can directly minimize the negative log likelihood of this model

$$\mathcal{L}_r(\psi, \mathcal{D}) = -E_{(\bar{x},\bar{y}_w,\bar{y}_l)\sim\mathcal{D}}\left[\log p(\bar{y}_w \succ \bar{y}_l)\right]$$
$$= -E_{(\bar{x},\bar{y}_w,\bar{y}_l)\sim\mathcal{D}}\left[\log \sigma\left(r_\psi([\bar{x};\bar{y}_w]) - r_\psi([\bar{x};\bar{y}_l])\right)\right]$$

- This gives us a relatively straightforward supervised learning problem (even if a pretty hard one) to *train the reward model*.

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO)

- PPO [Schulman et al. 2017] is the most common choice for RLHF
- Empirically provides several advantages of REINFORCE
  - Increased stability and reliability, reduction in gradient estimates variance, and faster learning
- But, has more hyper-parameters and requires to estimate the value function $v_\pi(s)$

$$v_{\pi_\theta(s)} = E_{\pi_\theta}[G_t \mid s]$$

PPO: Advantage Actor-critic

- PPO is an **advantage actor-critic** method
  - actor-critic: the learning objective includes an estimated value function to "critique" the policy (actor) actions
  - advantage: instead of optimizing directly using rewards like REINFORCE, updates rely on ***advantage***
- ***Advantage*** is the benefit of taking an action at a state relative to other actions at the same state

$$A_\pi(s,a) = \underbrace{r(s,a) + v_\pi(T(s,a))}_{q_\pi(s,a)^*} - v_\pi(s)$$

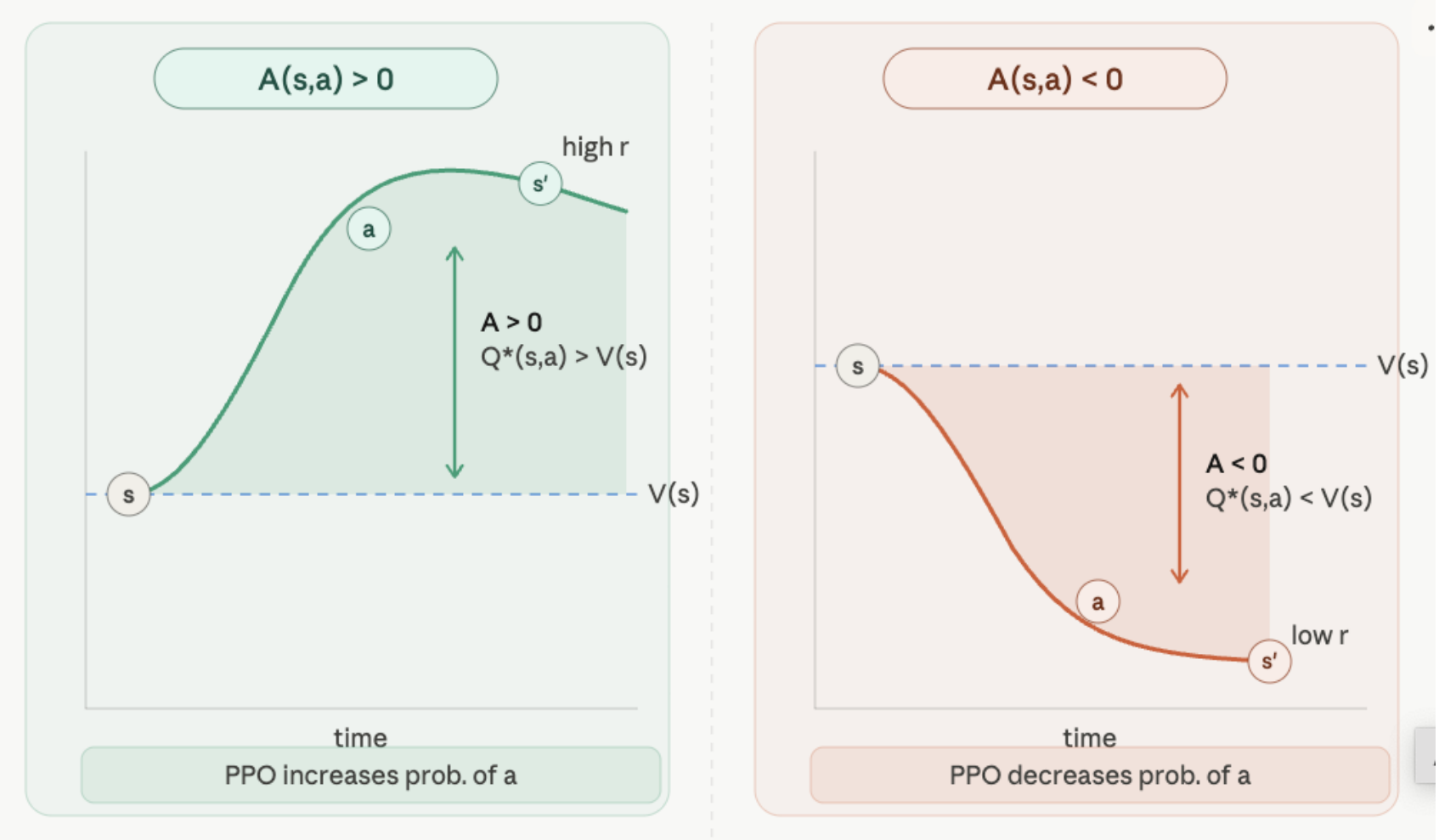
PPO: Advantage Actor-critic

- ***Advantage*** is the benefit of taking an action at a state relative to other actions at the same state

$$A_\pi(s,a) = \underbrace{r(s,a) + v_\pi(T(s,a))}_{q_\pi(s,a)^*} - v_\pi(s)$$

(what you got + what you expect next) − what you originally expected

Advantage (illustration)



PPO: Reward Maximization Under Penalty

- PPO balances between
  - Increasing the expected reward by significantly changing the policy
  - Keeping the policy as close as possible to the original policy to maintain stability

$$\arg\max_{\theta} E_{\pi}\left[\frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}(s,a)\right.$$

PPO -KL

- Penalizes the KL-divergence in the objective function
- It is based on optimizing a penalized objective:

$$\arg\max_{\theta} E_{\pi}\left[\frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}(s,a) - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_{\theta}(\cdot \mid s_t)]\right]$$

PPO-Clip Objective (Alternative)

- PPO-clip typically updates the policy by taking multiple steps of (usually minibatch) SGD to maximize the objective L. Here L is given by (where ϵ is a small hyperparameter specifies how far

$$L(s, a, \theta_k, \theta) = \min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a),\ \ \text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}, 1-\epsilon, 1+\epsilon\right) A^{\pi_{\theta_k}}(s, a)\right)$$

What does it mean when the ratio is really big? Or really small?

Reference: https://spinningup.openai.com/en/latest/algorithms/ppo.html

PPO Practicality

- PPO is notoriously complex to work with
  - It requires learning a separate value function
  - Two internal optimizations loops — learning rate? number of epochs? optimizer?
  - So has quite a few hyper-parameters, and turns out PPO is very sensitive to them
  - See: The 37 Implementation Details of Proximal Policy Optimization

RLHF Examples — Llama 2

- Llama 2 is a family of LLMs from Meta
- Ranging 7-70B parameters
- RLHF and reward model designs were customized to some degree, but overall follow the conventional recipe
- Meta wrote a report that provides relatively detailed insights into some key steps in the process

RLHF Examples — Llama 2 Performance

- The reward model was trained in an iterative process
- Intermediate models were used to pick examples to annotate for preferences for later models (why?)

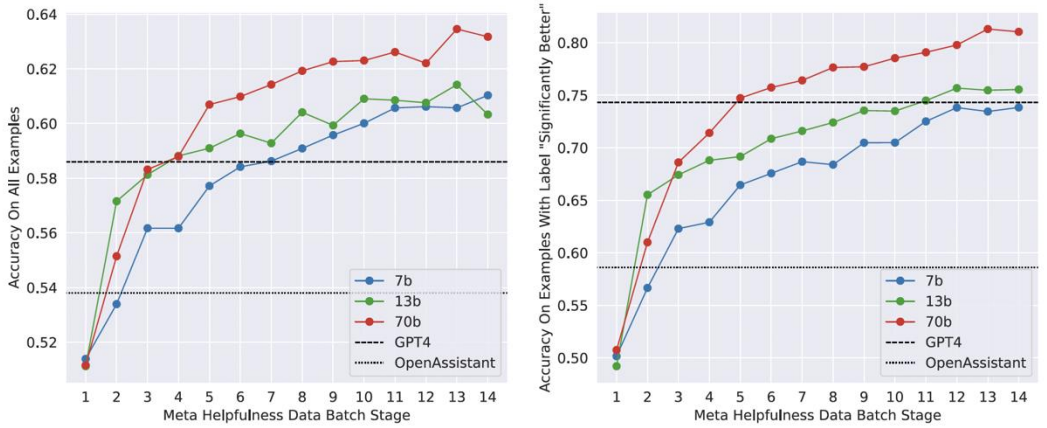


Figure 6: Scaling trends for the reward model. More data and a larger-size model generally improve accuracy, and it appears that our models have not yet saturated from learning on the training data.

RLHF with PPO Takeaways

- A pretty complex process
- Hard to get it to work — both reward modeling and RL
- Very costly — both compute and data annotation
- But, works really well
- Basically all SOTA models at this point go through RLHF
- There are a lot of tricky implementation details

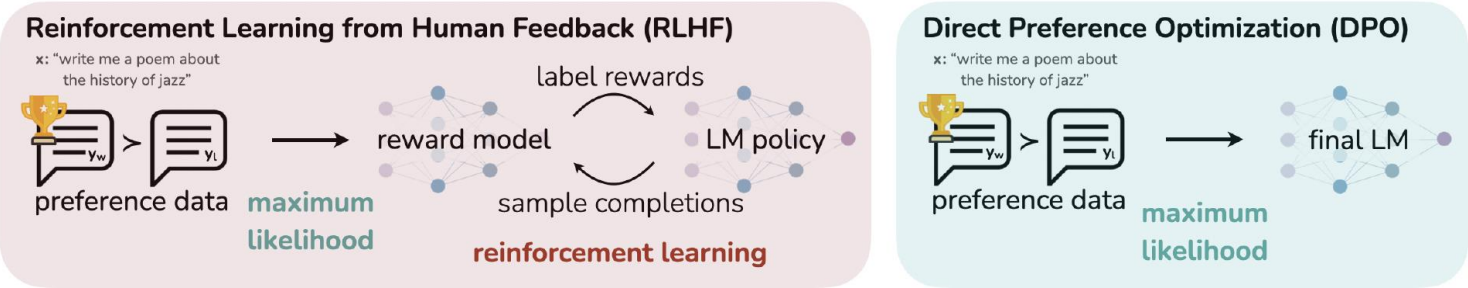
RLHF Revisit

- The entire process is based on fixed annotated data $\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}_w^{(i)}, \boldsymbol{y}_l^{(i)})\}_{i=1}^N$
- There is no other source of learning signal
- Can we just think of the entire process as a supervised learning problem?

Direct Policy Optimization (DPO)

Direct Policy Optimization (DPO)

- Adopt an alternative ***offline RL*** setup
  - Offline RL uses a static set of trajectories with rewards, rather than new trajectories generated during learning (like we saw in REINFORCE and PPO)
- Restrict the reward to a specific form
- Combine the reward learning objective with an RL objective to directly optimize a *policy*
  - Recall: what's our *policy* here?



DPO Objectives

- DPO starts with a very similar RL objective to PPO

$$\arg\max_\theta E_{\bar{x}\sim\mathcal{D},\bar{y}\sim\pi_\theta(\bar{y}|\bar{x})}\left[r(\bar{x},\bar{y}) - \beta \mathrm{KL}[\pi_\theta(\bar{y}\,|\,\bar{x}), \pi_{\text{ref}}(\bar{y}\,|\,\bar{x})]\right]$$

Maximize the expected reward according to our prompt data and policy

Penalize for the distribution getting further from the pre-RL distribution

Recall PPO:

$$\arg\max_\theta E_\pi\left[\frac{\pi_\theta(a_t\,|\,s_t)}{\pi_{\theta_{\text{old}}}(a_t\,|\,s_t)}\hat{A}(s,a) - \beta \mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot\,|\,s_t), \pi_\theta(\cdot\,|\,s_t)]\right]$$

DPO Derivation

- We can express the reward function:

$$r(\bar{x}, \bar{y}) = \beta \log \frac{\pi^*(\bar{y} \mid \bar{x})}{\pi_{\text{ref}}(\bar{y} \mid \bar{x})} + \beta \log Z(\bar{x})$$

- Why is this important?
- Remember: the RLHF reward is the scoring function s(.) in the Bradley-Terry preference model
  - $p(a \succ b) = \sigma(s(a) - s(b))$

DPO Derivation

- We can express the reward function:

$$r(\bar{x}, \bar{y}) = \beta \log \frac{\pi^*(\bar{y} \mid \bar{x})}{\pi_{\text{ref}}(\bar{y} \mid \bar{x})} + \beta \log Z(\bar{x})$$

- So we can simply plug the above reward to the Bradley-Terry model:

$$\begin{aligned} p(\bar{y}_w \succ \bar{y}_l) &= \sigma\left(\beta \log \frac{\pi^*(\bar{y}_w \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_w \mid \bar{x})} + \beta \log Z(\bar{x}) - \beta \log \frac{\pi^*(\bar{y}_l \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_l \mid \bar{x})} - \beta \log Z(\bar{x})\right) \\ &= \sigma\left(\beta \log \frac{\pi^*(\bar{y}_w \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_w \mid \bar{x})} - \beta \log \frac{\pi^*(\bar{y}_l \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_l \mid \bar{x})}\right) \end{aligned}$$

DPO Derivation

- If we use π_θ instead of π^*, we directly get a negative log-likelihood loss to optimize:

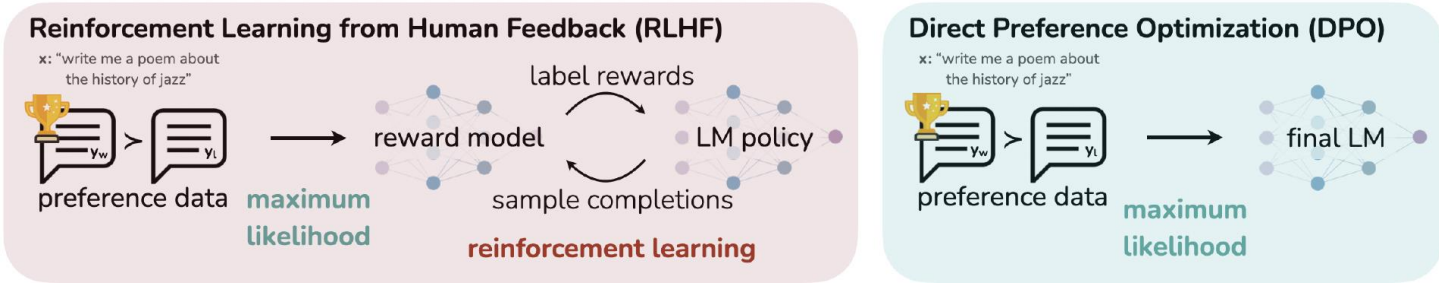
$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \prod_{(\bar{x},\bar{y}_w,\bar{y}_l)\in\mathcal{D}} p(y_w \succ y_l)$$

$$= -E_{(\bar{x},\bar{y}_w,\bar{y}_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(\bar{y}_w \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_w \mid \bar{x})} - \beta \log \frac{\pi_\theta(\bar{y}_l \mid \bar{x})}{\pi_{\text{ref}}(\bar{y}_l \mid \bar{x})}\right)\right]$$

Increase likelihood of preferred example

Decrease likelihood of dispreferred example

Comparing DPO to PPO



- RLHF (PPO) and DPO both can be used to compute rewards
  - RLHF (PPO): explicitly learns a reward model $r_\psi(x, y)$
  - DPO: we can compute the reward using the base and fine-tuned models $r(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$
- Reward model is a key, and there is still a lot of room for improvement (as is shown in Llama 2 results)

LLMs Alignment Key Takeaways

- RLHF is an essential, but complex and compute-intensive process to make expressive LLMs useful as multitasking agents.
- It presents a very restricted instance of RL (basically a bandit problem), even though it uses relatively advanced algorithms
- *Data is the key to the process*, and it requires careful curation and annotation
- There are supervised (offline RL) approaches that can get similar or even equal results (e.g., DPO)
- A lot of active research in this area

Presentation

- Give 1-5 (5=best) rating for the presetnation

Go to

join.iClicker.com

LCKW